

Assignment 1

Process Abstractions, Scheduling and Performance Analysis

Vedansh Garg

Contents

1	Process Abstraction & POSIX API	2
1.1	1.1 State Machines Under Pressure	2
1.2	1.2 Deep fork Analysis	3
2	Scheduling Algorithms	4
2.1	2.1 Multi-Process Workload	4
2.2	2.2 Theoretical Bounds	5
3	Performance Metrics	5
3.1	3.1	5
3.2	3.2	6

1 Process Abstraction & POSIX API

1.1 1.1 State Machines Under Pressure

(a)

When the timer interrupt occurs:

1. CPU switches to kernel mode.
2. Hardware saves minimal context.
3. Kernel saves registers into PCB.
4. Running process state changes Running $\rightarrow$ Ready.
5. PCB inserted into ready queue.
6. Scheduler examines ready queue.
7. Context switch loads next process context.
8. Selected process becomes Running.
9. Return-from-interrupt restores registers.

(b)

Invalid transitions:

- Running $\rightarrow$ Terminated: read() does not terminate process.
- Terminated $\rightarrow$ Blocked: impossible.
- Blocked can only be entered by a live process.

Correct sequence:

Running $\rightarrow$ Blocked $\rightarrow$ Ready $\rightarrow$ Running

Explanation:

1. read() on empty socket causes blocking.
2. NIC interrupt signals data arrival.
3. Kernel wakes process and places it in Ready queue.
4. Scheduler dispatches it.

(c)

A zombie retains:

- PID
- Exit status
- Resource accounting information

until parent calls `wait()` or `waitpid()`.

If parent exits first, orphan adoption by `init/systemd` occurs and the zombie is eventually reaped.

1.2 1.2 Deep fork Analysis

- a) 2^n processes are created because the number of processes doubles on each iteration (as the child is not terminated inside the loop). Every existing process creates one additional process at each `fork()`.

In the given code, only 3 processes are created because the second `fork()` is run inside the conditional block (`if (rc1 == 0)`), so a third process is created only by the first child process.

- b) **Variable Values across Processes:**

- **C:** $g = 110, \quad x = 105$
- **B:** $g = 10, \quad x = 5$
- **A:** $g = 10, \quad x = 5$

Whenever a child process is created, it makes its own copy of the parent's memory, so any modifications made within the child process do not propagate to its parent.

- c) Any order other than $C \rightarrow B \rightarrow A$ is impossible because of the `wait()` commands.
- d) (i) **No**, even the heap memory is duplicated after calling `fork()`.

- (ii) A **use-after-free** bug can occur in such cases. It occurs when memory is accessed after being deallocated.

Explanation – Copy-on-Write (CoW): It is a resource management technique where the system delays duplicating data until a modification is requested. Instead of copying data immediately, the original and the new process share the same memory page.

- (iii) Process C becomes an **orphan process** but it continues to run normally. In such a case, the OS re-parents C to the `init` process. If this situation happens repeatedly, zombie processes may accumulate and exhaust the resources of the PCB table.

2 Scheduling Algorithms

2.1 Multi-Process Workload

Workload:

Process	Arrival	Burst	Priority
P1	0	10	3
P2	2	4	1
P3	4	6	2
P4	6	2	4
P5	8	8	2

SRTF Preemptions

At $t = 2$: P2 arrives with burst 4 while P1 has 8 remaining. Therefore P1 is preempted.

At $t = 6$: P4 arrives with burst 2 while P3 has 4 remaining. Therefore P3 is preempted.

Round Robin ($q = 3$)

Gantt chart:

0-3 P1
3-6 P2
6-9 P1
9-12 P3
12-14 P4
14-15 P2
15-18 P5
18-21 P1
21-24 P3
24-27 P5
27-28 P1
28-30 P5

Completion times:

Process	Completion	Turnaround
P1	28	28
P2	15	13
P3	24	20
P4	14	8
P5	30	22

Wait times:

$$W_1 = 28 - 10 = 18$$

$$W_2 = 13 - 4 = 9$$

$$W_3 = 20 - 6 = 14$$

$$W_4 = 8 - 2 = 6$$

$$W_5 = 22 - 8 = 14$$

Average wait:

$$\frac{18 + 9 + 14 + 6 + 14}{5} = 12.2$$

Average turnaround:

$$\frac{28 + 13 + 20 + 8 + 22}{5} = 18.2$$

Non-preemptive SJF

Order:

$$P1 \rightarrow P4 \rightarrow P2 \rightarrow P3 \rightarrow P5$$

Wait times:

$$0, 4, 10, 12, 14$$

Average wait:

$$8 \text{ ms}$$

Average turnaround:

$$14 \text{ ms}$$

2.2 Theoretical Bounds

- a) The average wait time for SJF $= 0 + b_1 + \frac{b_1+b_2}{3} = \frac{2b_1+b_2}{3}$.

For any preemptive process, avg. wait time $= 0 + b_i + \frac{b_i+b_j}{3} = \frac{2b_i+b_j}{3}$.

This expression is minimized for $i = 1$ and $j = 2$ given that $b_1 \leq b_2 \leq b_3$.

- b) (i) **SRTF**: Avg. response time $= \frac{0+s+2s+\dots+(k-1)s+ks}{k+1} = \frac{ks}{2}$

RR: Avg. response time $= \frac{0+q+2q+\dots+(k-1)q+kq}{k+1} = \frac{kq}{2}$

Round Robin (RR) always produces a lower average response time than SRTF for $q < s$.

3 Performance Metrics

3.1

Given:

Process	Start	Finish
P1	0	12
P2	12	15
P3	15	22
P4	22	24
P5	24	30

Wait times:

$$(0, 12, 15, 22, 24)$$

Turnaround times:

$$(12, 15, 22, 24, 30)$$

Makespan:

$$30 - 0 = 30$$

Jain Index:

$$J = \frac{(73)^2}{5(1429)} = \frac{5329}{7145} \approx 0.746$$

3.2 3.2

- a) **SRTF**, because many small jobs finish almost instantly and long jobs get postponed repeatedly.

Example Workload:

Process	Arrival Time	Burst Time
P_1	0	100
P_2	1	1
P_3	2	1

Consequently, P_2 and P_3 experience 0 wait time while P_1 waits for as long as these shorter processes keep arriving.

- b) A scheduler can minimize Makespan by maximizing CPU utilization and completing large amounts of work continuously, even if this delays many smaller processes. Thus, the total workload finishes earlier, producing a lower Makespan, while individual processes may experience longer waits, increasing the average wait time.

Scheduler Γ (Gamma) likely favors throughput-oriented execution, reducing total completion time but sacrificing responsiveness for some jobs. Makespan is generally not a good metric for interactive workloads because users care about latency and responsiveness rather than total batch completion time. An interactive system may have excellent Makespan while still producing noticeable delays for individual user requests.

- c) (i) For scenario (i), I will surely choose **Gamma** (Γ) as it has the lowest Makespan, thus maximizing overall throughput.
- (ii) For scenario (ii), I will choose **Delta** (Δ) because, for a real-time chat application, latency and responsiveness are paramount, and the user experience is heavily affected by the worst-case scenario. Delta is optimal here since it yields the lowest P90 and P99 wait times.